Université de Tunis El Manar
Faculté des Sciences de Tunis

Apprentissage par renforcement

Théorie, algorithmes et étude expérimentale
sur un entrepôt simulé

Réalisatrices : Bouzidi Mariam
Bouhouch Eya
Lajmi Sahar

Encadrant : Prof. Samir Toumi

Projet réalisé en avril 2024 — version révisée et enrichie, octobre 2026

Résumé

Ce rapport présente l'apprentissage par renforcement (AR) : cadre des processus de décision markoviens (MDP), algorithmes classiques (Q-learning, SARSA, gradient de politique), applications et défis. La partie théorique est complétée par une étude expérimentale sur un entrepôt de 11×11 cases, assez petit pour être résolu *exactement* par programmation dynamique. Cette solution exacte sert de référence pour mesurer, sur 20 graines aléatoires, la qualité de chaque algorithme, au lieu de juger un seul entraînement à l'œil. Nous montrons que (i) le Q-learning trouve les plus courts chemins rapidement et pour une large plage d'hyper-paramètres, (ii) SARSA est nettement plus fragile, pour une raison que la solution exacte explique : l'exploration est dangereuse dans cet entrepôt et SARSA apprend la valeur d'une politique qui explore, (iii) la politique converge bien avant la fonction de valeur, et (iv) REINFORCE fonctionne mais demande environ cinq fois plus d'épisodes, le terme de base (*baseline*) réduisant nettement la variance.

Table des matières

1 Introduction à l'apprentissage par renforcement **3**
1.1 Définition . . . 3
1.2 Distinction entre les types d'apprentissage . . . 3
1.3 Historique et évolution . . . 3

2 L'écosystème de l'AR **4**
2.1 Acteurs . . . 4
2.2 Fondements conceptuels . . . 4

3 Processus de décision markovien **4**
3.1 Définition . . . 4
3.2 Retour et fonctions de valeur . . . 4
3.3 Équations de Bellman et politique optimale . . . 5
3.4 Résolution par programmation dynamique . . . 5

4 Algorithmes classiques **5**
4.1 Q-learning (hors politique) . . . 5
4.2 SARSA (sur politique) . . . 6
4.3 Gradient de politique (REINFORCE) . . . 6

5 Étude expérimentale : un entrepôt simulé **7**
5.1 L'environnement . . . 7
5.2 Vérité terrain . . . 7
5.3 Q-learning contre SARSA . . . 8
5.4 Pourquoi SARSA peine : explorer est dangereux . . . 9
5.5 Sensibilité à α, ε et γ . . . 9
5.6 Dynamique stochastique . . . 10

5.7 Gradient de politique : REINFORCE 10
5.8 Limites 11

6 Domaines d'application **11**

7 Problèmes typiques résolus par l'AR **11**
7.1 Jeux : AlphaGo 11
7.2 Contrôle robotique 11
7.3 Gestion de ressources 12

8 Défis et complexités **12**
8.1 Le dilemme exploration–exploitation 12
8.2 Les états continus 12
8.3 Autres difficultés 12

9 Avenir de l'AR **12**

10 Conclusion **13**

1 Introduction à l'apprentissage par renforcement

1.1 Définition

L'apprentissage par renforcement est le cadre dans lequel un *agent* apprend à prendre des décisions séquentielles en interagissant avec un *environnement*, dans le but de maximiser une récompense cumulée. À chaque instant, l'agent observe un état, choisit une action, reçoit une récompense et l'environnement passe dans un nouvel état. L'agent n'a pas de professeur : il ajuste son comportement par essais et erreurs, en fonction des récompenses obtenues.

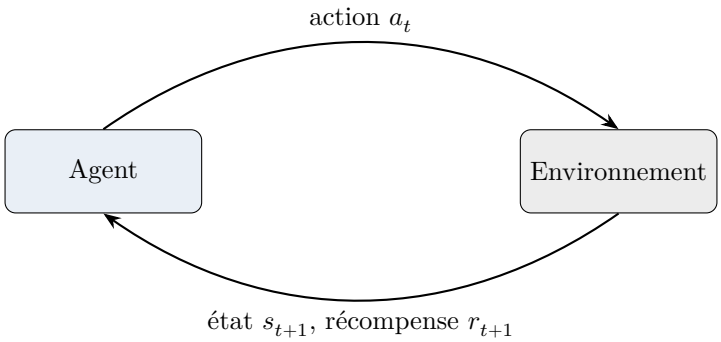


Fig. 1 : Boucle d'interaction agent–environnement.

1.2 Distinction entre les types d'apprentissage

	Supervisé	**Non supervisé**	**Par renforcement**
Données	exemples étiquetés (x, y)	données brutes x	expérience (s, a, r, s') générée par l'agent
Signal	la bonne réponse	aucun	récompense, souvent retardée
Objectif	prédire y	découvrir une structure	maximiser la récompense cumulée
Particularité	données indépendantes	–	les actions de l'agent influencent les données futures

La différence essentielle est l'absence de supervision explicite : on ne dit jamais à l'agent quelle action était la bonne, seulement si le résultat était bon, parfois bien plus tard (problème d'*attribution de crédit*). De plus, les données dépendent de la politique de l'agent, ce qui crée le dilemme exploration–exploitation (section 8).

1.3 Historique et évolution

Les racines de l'AR sont la programmation dynamique de Bellman (années 1950), la psychologie de l'apprentissage par essais et erreurs, et la théorie du contrôle optimal. Les jalons marquants sont : les méthodes à différence temporelle de Sutton (1988), le Q-learning de Watkins (1989) et sa preuve de convergence (Watkins et Dayan, 1992), REINFORCE de Williams (1992), TD-Gammon de Tesauro (1995), puis l'arrivée de l'apprentissage profond avec DQN (Mnih et al., 2015) et la victoire d'AlphaGo contre Lee Sedol en mars 2016 (Silver et al., 2016).

2 L'écosystème de l'AR

2.1 Acteurs

- **Agent** : entité qui décide. Il cherche une politique qui maximise la récompense cumulée.
- **Environnement** : tout ce qui est extérieur à l'agent. Il réagit aux actions en produisant un nouvel état et une récompense.
- **Récompense** : signal scalaire qui indique la qualité immédiate d'une transition. Elle *définit* l'objectif : une récompense mal conçue donne un agent qui optimise la mauvaise chose (section 9).

2.2 Fondements conceptuels

- **État** s : description de la situation, observable ou partiellement observable.
- **Action** a : choix de l'agent ; l'ensemble des actions forme l'espace d'actions (discret ou continu).
- **Politique** π : règle de décision, déterministe $a = \pi(s)$ ou stochastique $\pi(a \mid s)$.

3 Processus de décision markovien

3.1 Définition

Un MDP est un tuple $\langle S, A, P, R, \gamma \rangle$ où S est l'ensemble (fini) des états, A celui des actions, $P(s' \mid s, a)$ la fonction de transition, $R(s, a, s')$ la fonction de récompense et $\gamma \in [0, 1)$ le facteur d'actualisation.

Propriété de Markov. La probabilité d'atteindre s' ne dépend que de l'état courant et de l'action, pas de l'historique :

$$P(s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s' \mid s_t, a_t).$$

3.2 Retour et fonctions de valeur

Le *retour* à partir de l'instant t est la somme actualisée des récompenses futures :

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1} = r_{t+1} + \gamma \, G_{t+1}. \tag{1}$$

Le facteur γ traduit la préférence pour les récompenses proches ; avec $\gamma < 1$ et des récompenses bornées, la somme converge. Les fonctions de valeur d'une politique π sont

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \tag{2}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, \, a_t = a]. \tag{3}$$

3.3 Équations de Bellman et politique optimale

Une politique π^* est optimale si $V^{\pi^*}(s) \geq V^{\pi}(s)$ pour tout s et toute π. Ses fonctions de valeur V^* et Q^* vérifient les équations d'optimalité de Bellman :

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\Big], \tag{4}$$

$$V^*(s) = \max_a Q^*(s,a), \qquad \pi^*(s) \in \arg\max_a Q^*(s,a). \tag{5}$$

La valeur d'un état est donc celle de sa meilleure action, et la politique optimale s'obtient en choisissant, dans chaque état, l'action qui maximise Q^*.

3.4 Résolution par programmation dynamique

Lorsque le modèle (P, R) est connu et que S, A sont finis, on résout (4) par *itération de la valeur* : on applique répétitivement l'opérateur d'optimalité de Bellman. Cet opérateur est une contraction de rapport γ pour la norme infinie, donc l'itération converge vers l'unique point fixe V^* (théorème de Banach).

Algorithme 1 : Itération de la valeur

```
V ← 0 ;
répéter
    pour chaque s ∈ S non terminal, a ∈ A faire
        Q(s,a) ← Σ_{s'} P(s' | s,a) [R(s,a,s') + γV(s')] ;
    V_new(s) ← max_a Q(s,a) ;
jusqu'à max_s |V_new(s) − V(s)| < τ ;
retourner Q, π(s) = arg max_a Q(s,a)
```

Les méthodes d'AR traitent le cas où P et R sont *inconnus* : elles apprennent à partir d'échantillons de transitions.

4 Algorithmes classiques

Les deux premiers algorithmes apprennent $Q(s,a)$ par *différence temporelle* (TD) et utilisent une politique de comportement ε-greedy : avec probabilité ε on tire une action au hasard (exploration), sinon on prend $\arg\max_a Q(s,a)$ (exploitation).

4.1 Q-learning (hors politique)

Après chaque transition (s, a, r, s') :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{cible}} - Q(s,a)\Big], \tag{6}$$

avec α le taux d'apprentissage, et un terme d'amorçage nul si s' est terminal. La cible est un échantillon de l'équation (4). Le Q-learning est *hors politique* : il évalue la politique gloutonne quelle que soit la politique de comportement. Dans le cas tabulaire, $Q \to Q^*$ avec probabilité 1 si toutes les paires (s,a) sont visitées infiniment souvent et si α_t vérifie les conditions de Robbins–Monro ($\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$) [2]. Avec un α constant, comme dans nos expériences, l'estimation reste bruitée mais suit la cible.

4.2 SARSA (sur politique)

SARSA (*State, Action, Reward, State, Action*) utilise l'action a' réellement choisie par la politique de comportement :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma\, Q(s',a') - Q(s,a)\Big], \qquad a' \sim \varepsilon\text{-greedy}(Q(s',\cdot)). \tag{7}$$

SARSA est *sur politique* : il estime la valeur de la politique ε-greedy qu'il suit, *exploration comprise*. Avec ε constant, il converge donc vers la fonction Q^*_ε solution de

$$Q^*_\varepsilon(s,a) = \sum_{s'} P(s' \mid s,a)\Big[R + \gamma\Big((1-\varepsilon)\max_{a'} Q^*_\varepsilon(s',a') + \varepsilon\,\tfrac{1}{|A|}\sum_{a'} Q^*_\varepsilon(s',a')\Big)\Big], \tag{8}$$

et non vers Q^* ; il converge vers Q^* si $\varepsilon \to 0$ de façon adaptée (condition GLIE) [4]. C'est la raison pour laquelle SARSA est plus « prudent » que le Q-learning, ce que nous mesurons à la section 5.

Algorithme 2 : Q-learning et SARSA avec politique ε-greedy

```
Initialiser Q(s,a) ← 0 ;
pour chaque épisode faire
    s ← état initial ; a ← ε-greedy(Q(s,·)) ;
    tant que s non terminal faire
        exécuter a, observer r, s' ;
        si Q-learning alors
            y ← r + γ max_{a'} Q(s',a')
        sinon
            choisir a' ← ε-greedy(Q(s',·)) ; y ← r + γQ(s',a')
        y ← r si s' est terminal ;
        Q(s,a) ← Q(s,a) + α (y − Q(s,a)) ;
        s ← s' ; a ← a'  (Q-learning : a' ← ε-greedy(Q(s',·))) ;
```

4.3 Gradient de politique (REINFORCE)

Au lieu d'apprendre une fonction de valeur, on paramètre directement la politique, $\pi_\theta(a \mid s)$ (par exemple une softmax $\pi_\theta(a \mid s) \propto e^{\theta_{s,a}}$ dans le cas tabulaire), et on maximise l'espérance du retour :

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G_0], \qquad P_\theta(\tau) = P(s_1)\prod_{t=1}^{|\tau|-1} \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t).$$

La dynamique P ne dépend pas de θ : l'astuce de la dérivée logarithmique donne le théorème du gradient de politique,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(G_t - b(s_t))\Big], \tag{9}$$

valable pour toute *baseline* $b(s)$ indépendante de l'action. La baseline ne biaise pas le gradient mais réduit sa variance. L'algorithme REINFORCE [3] fait une montée de gradient stochastique avec cette estimation : $\theta \leftarrow \theta + \alpha\,(G_t - b(s_t))\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)$. Pour une softmax tabulaire, $\nabla_{\theta_{s,\cdot}} \log \pi_\theta(a \mid s) = \mathbf{1}_a - \pi_\theta(\cdot \mid s)$.

Ces méthodes s'appliquent naturellement aux espaces d'actions continus et aux politiques stochastiques, au prix d'une variance élevée et d'une grande consommation d'échantillons.

5 Étude expérimentale : un entrepôt simulé

Un seul entraînement jugé « à l'œil » ne prouve pas grand-chose. Nous avons donc choisi un problème assez petit pour être résolu *exactement*, afin de mesurer l'écart de chaque algorithme à l'optimum, sur 20 graines aléatoires. Tout le code est dans le dépôt (`rl_warehouse.py` et le notebook) et reproduit exactement les chiffres ci-dessous.

5.1 L'environnement

Un robot doit rejoindre la zone d'emballage d'un entrepôt de 11×11 cases (figure 2). C'est un MDP avec $|S| = 121$ états, quatre actions (haut, droite, bas, gauche), et la récompense reçue en entrant dans une case : $+100$ à la zone d'emballage, -1 sur chaque case d'allée (donc plus court est mieux), -100 sur un rayonnage (l'épisode s'arrête). Les 56 cases d'allée servent de points de départ (tirés uniformément), et $\gamma = 0.9$. Les états terminaux ont une valeur nulle et ne servent jamais d'amorçage. Une variante stochastique ajoute une probabilité *slip* de remplacer l'action choisie par une action aléatoire.

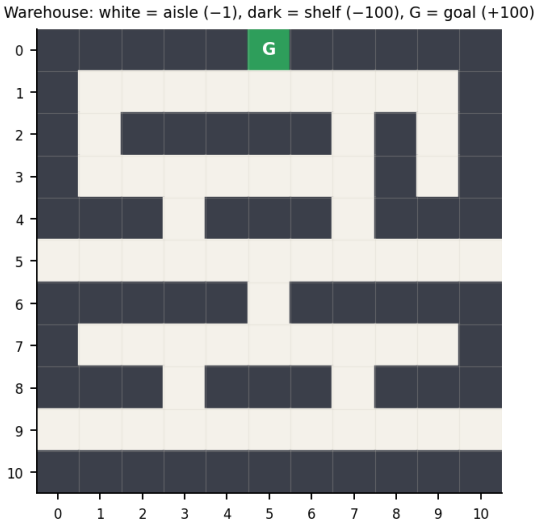


Fig. 2 : L'entrepôt : allées en blanc, rayonnages en gris foncé, zone d'emballage (G) en vert.

5.2 Vérité terrain

L'itération de la valeur (algorithme 1) converge en 19 balayages. Comme contrôle indépendant, un parcours en largeur (BFS) donne la longueur exacte du plus court chemin depuis chaque case (au plus 18 pas). La politique gloutonne de Q^* suit un plus court chemin depuis les 56 départs, ce qui valide les deux oracles. La figure 3 montre V^*, la politique optimale et trois plus courts chemins.

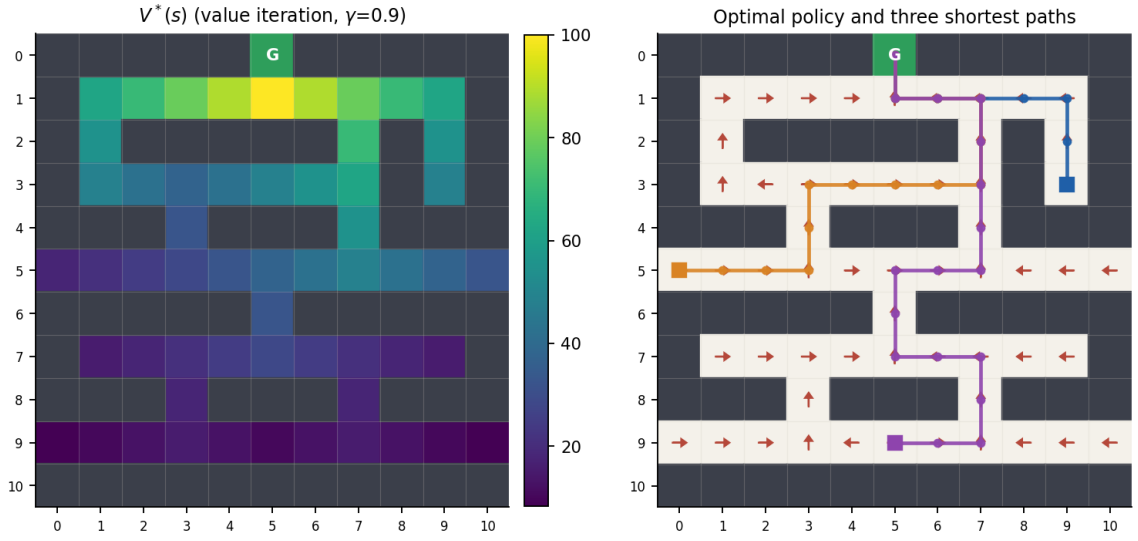


Fig. 3 : Gauche : V^*. Droite : politique optimale et trois plus courts chemins.

5.3 Q-learning contre SARSA

Chaque configuration est entraînée 2000 épisodes avec 20 graines ($\varepsilon = 0.1$). Toutes les 50 épisodes, la politique gloutonne est évaluée exactement depuis les 56 départs ; on mesure aussi l'erreur de $V(s) = \max_a Q(s,a)$ par rapport à la *propre cible* de l'algorithme (Q^* pour le Q-learning, Q^*_ε pour SARSA, toutes deux calculées exactement).

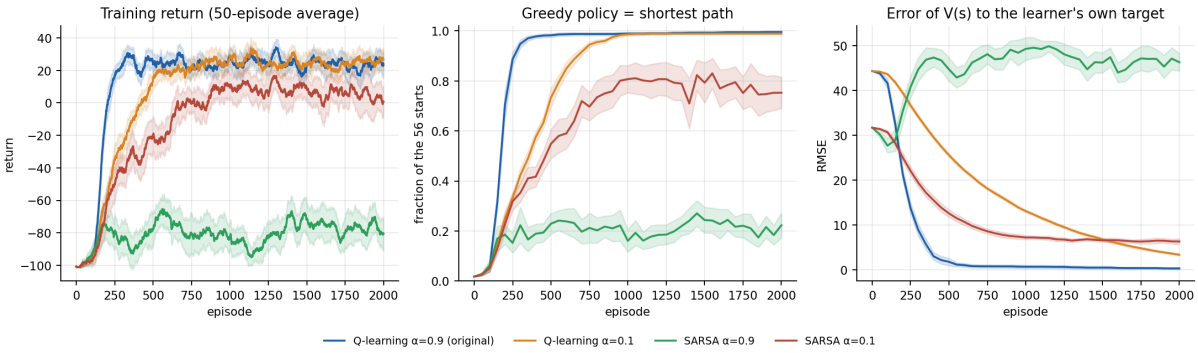


Fig. 4 : Moyenne sur 20 graines et intervalle de confiance à 95 %. Gauche : retour d'entraînement. Centre : fraction des départs dont le chemin glouton est un plus court chemin. Droite : erreur de V à la cible.

Configuration	Atteint le but	Plus court chemin	Épisodes pour 90 %
Q-learning, $\alpha = 0.9$ (version initiale)	100 ± 0 %	99.6 ± 0.3 %	300
Q-learning, $\alpha = 0.1$	100 ± 0 %	98.9 ± 0.5 %	700
SARSA, $\alpha = 0.1$	82.1 ± 5.3 %	75.3 ± 6.3 %	jamais
SARSA, $\alpha = 0.9$	24.2 ± 5.9 %	22.3 ± 4.9 %	jamais

Tab. 1 : Politique gloutonne finale (moyenne $\pm$ IC 95 % sur 20 graines).

Le Q-learning trouve les plus courts chemins rapidement, quel que soit α ; SARSA est bien moins bon. Un point à retenir : la **politique converge bien avant la fonction de valeur**, car seul l'ordre des quatre actions compte.

5.4 Pourquoi SARSA peine : explorer est dangereux

Ce n'est pas une erreur de code, et la solution exacte le montre. SARSA apprend la valeur de la politique qu'il *suit*, exploration comprise. Or, dans cet entrepôt, presque toutes les allées sont bordées de rayonnages, donc un pas aléatoire est souvent fatal. La valeur exacte V_ε^* est **négative sur 28 des 56 cases** (minimum -24), alors que $V^* > 0$ partout : un agent qui explore s'écrase, en espérance, avant d'atteindre le but (figure 5).

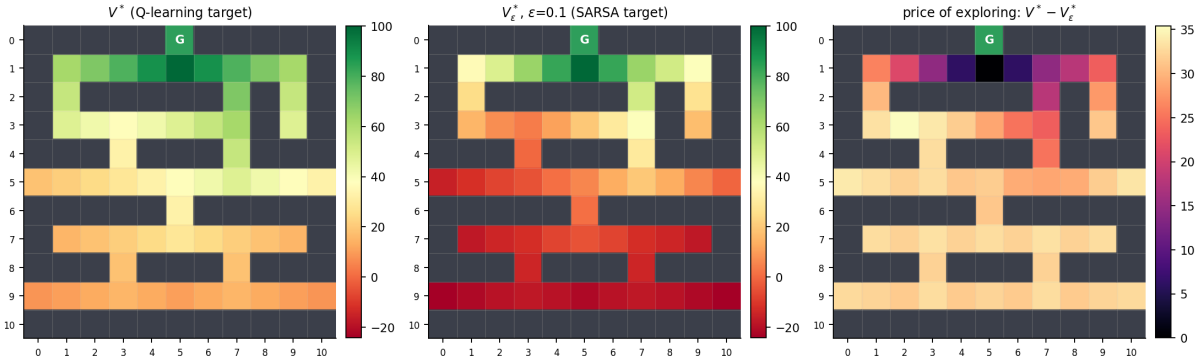


Fig. 5 : Cibles du Q-learning (V^*) et de SARSA (V_ε^*, $\varepsilon = 0.1$) ; à droite, le « prix de l'exploration ».

La politique gloutonne de Q_ε^* est pourtant un plus court chemin depuis les 56 départs : la *cible* de SARSA est bonne, c'est son *estimation* qui est bruitée. Les valeurs des actions y diffèrent de quelques unités, et un seul échantillon à -100 déplace l'estimation de 100α. L'amorçage par max du Q-learning ignore ce que l'exploration ferait ensuite, et donne des estimations bien moins bruitées. Les échecs de SARSA sont des *boucles* (18 % des couples départ/graine pour $\alpha = 0.1$), jamais des collisions. C'est le compromis biais–variance classique entre cibles sur et hors politique.

5.5 Sensibilité à α, ε et γ

La figure 6 donne la fraction de chemins optimaux après 1000 épisodes (8 graines par case). Le Q-learning est robuste presque partout ; il ne souffre que lorsque ε est grand *et* α petit (par exemple 42 % pour $\alpha = 0.05$, $\varepsilon = 0.4$). SARSA n'est bon que dans le coin α et ε petits (96–98 % pour $\alpha \le 0.1$, $\varepsilon = 0.02$, mais 13 % pour $\alpha = 0.9$, $\varepsilon = 0.1$). Le facteur γ a très peu d'effet sur le Q-learning (au moins 96 % même pour $\gamma = 0.5$) : avec une dynamique déterministe et un coût de -1 par pas, même de très petites valeurs actualisées ordonnent correctement les actions. SARSA est plus bas partout et non monotone en γ ; avec 8 graines nous ne cherchons pas à interpréter cette forme.

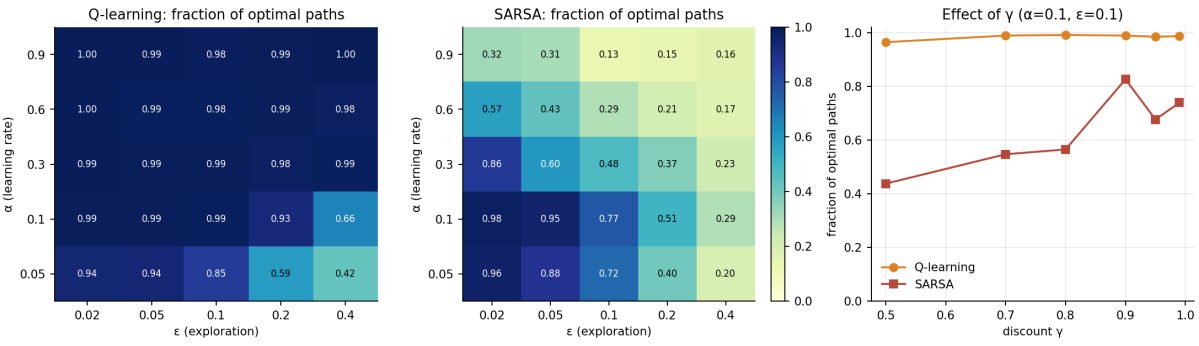


Fig. 6 : Sensibilité aux hyper-paramètres (fraction de plus courts chemins).

5.6 Dynamique stochastique

Avec probabilité *slip*, l'action exécutée est aléatoire. L'optimum exact vient alors de l'itération de la valeur avec le modèle complet, et les politiques sont évaluées par simulation (10 graines, 500 trajectoires).

Taux d'atteinte du but	slip 0	0.05	0.1	0.2
Optimum exact (programmation dynamique)	1.00	0.85	0.68	0.41
Q-learning entraîné avec bruit ($\alpha = 0.1$)	1.00	0.77	0.50	0.32
SARSA entraîné avec bruit ($\alpha = 0.1$)	0.74	0.68	0.53	0.29
Q-learning entraîné *sans* bruit, déployé avec bruit	1.00	0.82	0.67	0.43

Tab. 2 : Même avec la solution exacte, le but n'est pas toujours atteint : un pas aléatoire près d'un rayonnage est fatal.

Deux observations. D'abord, la politique optimale pour slip $= 0.1$ ne diffère de la politique de plus courts chemins que sur **1 case sur 56** : dans des couloirs bordés de rayonnages, le risque s'accumule par pas, donc le plus court chemin est aussi le plus sûr. C'est pourquoi une politique apprise sans bruit se transfère presque parfaitement. Ensuite, le Q-learning entraîné *dans* l'environnement bruité fait *moins bien* avec $\alpha = 0.1$ (50 %) ; plus d'épisodes n'aident pas (49 % après 20 000), car le plancher de bruit est fixé par α. Avec $\alpha = 0.02$, il retrouve l'optimum exact (66 % contre 68 %). Un α décroissant (Robbins–Monro) ferait les deux.

5.7 Gradient de politique : REINFORCE

Nous avons implémenté REINFORCE avec une politique softmax tabulaire ($\alpha = 0.002$, 10 000 épisodes, 10 graines), avec et sans baseline (moyenne mobile des retours observés depuis l'état).

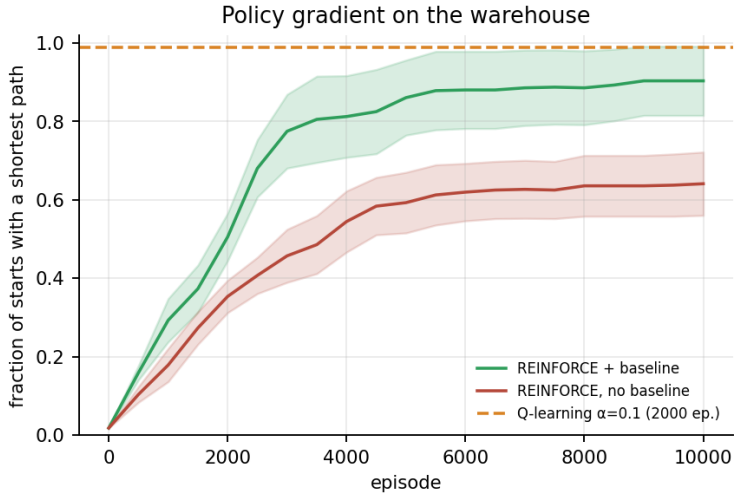


Fig. 7 : REINFORCE avec et sans baseline ; trait pointillé : Q-learning ($\alpha = 0.1$, 2000 épisodes).

Avec baseline, REINFORCE atteint un plus court chemin depuis 90.4 ± 8.8 % des départs (le but depuis 92.5 ± 8.4 %), contre 64.1 ± 8.1 % sans baseline : l'effet de réduction de variance est net. Il faut toutefois environ cinq fois plus d'épisodes que le Q-learning, et les résultats varient davantage d'une graine à l'autre.

5.8 Limites

L'environnement est minuscule et tabulaire : rien ici ne traite l'approximation de fonctions, et l'écart Q-learning/SARSA peut être différent ailleurs. Les hyper-paramètres des comparaisons principales viennent de l'étude de sensibilité et d'un petit essai pilote, sans réglage séparé par algorithme sur des graines réservées. Les intervalles de confiance sont des approximations normales sur 8 à 20 graines. Enfin, ε est constant : une décroissance (GLIE) est la voie standard pour que SARSA converge vers l'optimum, et n'a pas été explorée.

6 Domaines d'application

L'AR s'applique dès qu'il faut prendre des décisions séquentielles dont les conséquences sont différées. Quelques domaines où il est étudié ou déployé :

- **Systèmes de recommandation.** Recommander est un problème séquentiel : chaque suggestion influence les préférences et les données futures. Les bandits contextuels et l'AR sont employés pour personnaliser des contenus tout en continuant d'explorer de nouvelles options.
- **Chaîne d'approvisionnement.** Gestion des stocks, tournées, affectation de ressources : l'AR optimise des politiques sous demande incertaine.
- **Marketing et publicité en ligne.** Allocation d'enchères et de budgets, ajustée en continu selon le comportement des utilisateurs.
- **Énergie et bâtiments.** Régulation du chauffage, de la climatisation et de l'éclairage. DeepMind a rapporté en 2016 une réduction d'environ 40 % de l'énergie de refroidissement d'un centre de données de Google grâce à des recommandations issues de l'apprentissage automatique [11].
- **Robotique.** Navigation, manipulation d'objets. Un exemple marquant est une main robotique qui résout un Rubik's cube, entraînée en simulation puis transférée au monde réel [10].
- **Jeux vidéo et simulation.** Agents non joueurs adaptatifs, entraînement d'agents virtuels.

7 Problèmes typiques résolus par l'AR

7.1 Jeux : AlphaGo

En mars 2016, AlphaGo (DeepMind) bat le champion Lee Sedol 4 à 1 au jeu de go [6]. Le système combine des réseaux de neurones profonds (pour évaluer les positions et proposer des coups), une recherche arborescente de Monte-Carlo, et un entraînement en deux temps : apprentissage supervisé sur des parties humaines, puis amélioration par AR en jouant contre lui-même. Sa version suivante, AlphaGo Zero, apprend sans données humaines, uniquement par auto-jeu [7]. Ces résultats illustrent la puissance de l'AR couplé à l'apprentissage profond sur un espace d'états gigantesque.

7.2 Contrôle robotique

Un robot muni de capteurs (états) et d'actionneurs (actions) peut apprendre une tâche par AR à partir d'une récompense qui décrit le but. La difficulté est le coût des essais réels et le

fossé entre simulation et réalité ; on entraîne donc souvent en simulation, avec randomisation des paramètres physiques, avant de transférer [10].

7.3 Gestion de ressources

Contrôle du trafic, de l'énergie, allocation de ressources limitées : l'agent apprend des règles d'allocation qui s'adaptent à des conditions changeantes, avec des récompenses qui traduisent coûts et qualité de service.

8 Défis et complexités

8.1 Le dilemme exploration–exploitation

L'agent doit à la fois *exploiter* ce qu'il croit être la meilleure action et *explorer* d'autres actions pour découvrir de meilleures options. Trop exploiter fige l'agent sur une solution médiocre ; trop explorer gaspille des récompenses. Stratégies courantes : ε-greedy (action aléatoire avec probabilité ε, souvent décroissante), sélection softmax (Boltzmann), initialisation optimiste, et bornes de confiance supérieures (UCB). Notre étude montre que le bon niveau d'exploration dépend de l'algorithme : le Q-learning tolère ε jusqu'à 0.4 avec un α suffisant, alors que SARSA en subit directement le coût.

8.2 Les états continus

Une table $Q(s, a)$ n'est possible que pour un nombre fini et raisonnable d'états. Avec des états continus ou de grande dimension (positions d'un robot, pixels), on peut :

- **discrétiser**, ce qui souffre rapidement de la malédiction de la dimension ;
- **approximer** $Q_\theta(s, a)$ ou $\pi_\theta(a \mid s)$ par une fonction paramétrée, notamment un réseau de neurones. DQN [5] stabilise cet apprentissage par un *replay buffer* (échantillons décorrélés) et un *réseau cible* figé périodiquement ;
- pour les **actions continues**, utiliser des méthodes de gradient de politique et acteur–critique (par exemple PPO [8], SAC [9]).

L'association d'approximation de fonctions, d'amorçage et d'apprentissage hors politique (la « triade mortelle » [1]) peut diverger, d'où l'importance de ces stabilisations.

8.3 Autres difficultés

Efficacité en échantillons (millions d'interactions nécessaires), récompenses rares ou retardées, conception de la récompense, et observabilité partielle.

9 Avenir de l'AR

- **Tendances.** Convergence avec l'apprentissage profond, apprentissage hors ligne (à partir de données déjà collectées), apprentissage par imitation, méthodes basées sur un modèle du monde.

- **Éthique et interprétabilité.** Une récompense mal spécifiée peut être « détournée » (*reward hacking*) : l'agent maximise la lettre de la récompense, pas l'intention [12]. Dans les domaines sensibles, il faut comprendre et expliquer les décisions des agents, et borner les comportements dangereux pendant l'exploration (AR sûr).
- **Adaptation à l'adversité et à la dynamique.** Robustesse à des adversaires et à des environnements non stationnaires.
- **Applications sociales et économiques.** Éducation, santé, décision publique : des études exploratoires soulèvent des questions de responsabilité, de confidentialité et d'équité.

10 Conclusion

L'AR formalise l'apprentissage par essais et erreurs avec le cadre des MDP, dont les équations de Bellman donnent la solution exacte lorsque le modèle est connu. Lorsqu'il ne l'est pas, le Q-learning, SARSA et le gradient de politique apprennent à partir d'échantillons.

Notre étude expérimentale sur un entrepôt résoluble exactement a permis de *mesurer* plutôt que supposer : le Q-learning trouve les plus courts chemins (99.6 % des départs) de façon robuste ; SARSA est bien plus fragile, non par défaut de code mais parce que l'exploration est coûteuse dans cet environnement et que sa cible bruitée a de petites marges ; la politique converge avant les valeurs ; REINFORCE fonctionne avec une baseline mais demande beaucoup plus d'épisodes ; et en dynamique bruitée, le choix du pas d'apprentissage compte plus que le volume de données. Ces conclusions valent pour un problème tabulaire minuscule ; les étendre à l'approximation de fonctions (DQN, acteur–critique) est la suite naturelle de ce travail.

Références

[1] R. S. Sutton, A. G. Barto. *Reinforcement Learning : An Introduction*, 2e éd., MIT Press, 2018.

[2] C. J. C. H. Watkins, P. Dayan. Q-learning. *Machine Learning* 8, 279–292, 1992.

[3] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning* 8, 229–256, 1992.

[4] S. Singh, T. Jaakkola, M. L. Littman, C. Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning* 38, 287–308, 2000.

[5] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature* 518, 529–533, 2015.

[6] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature* 529, 484–489, 2016.

[7] D. Silver et al. Mastering the game of Go without human knowledge. *Nature* 550, 354–359, 2017.

[8] J. Schulman et al. Proximal policy optimization algorithms. arXiv :1707.06347, 2017.

[9] T. Haarnoja et al. Soft actor-critic. *ICML*, 2018.

[10] OpenAI et al. Solving Rubik's cube with a robot hand. arXiv :1910.07113, 2019.

[11] R. Evans, J. Gao. DeepMind AI reduces Google data centre cooling bill by 40%. Blog DeepMind, 2016.

[12] D. Amodei et al. Concrete problems in AI safety. arXiv :1606.06565, 2016.